

Завдання №1: Реалізація типу «множина»

1 Мета роботи

Опанувати способи представлення множин та їх ефективної реалізації, проаналізувати швидкість роботи операцій над множинами у заданій реалізації

2 Основне завдання

1. Реалізувати тип «множина» одним із способів, згідно із варіантом.
2. Реалізувати базові операції над множинами: включення елементу у множину (INSERT), видалення елементу з множини (DELETE), перевірку наявності елементу в множині (SEARCH), очистку множини (CLEAR).
3. Реалізувати додаткові операції над множинами: об'єднання множин (UNION), перетин множин (INTERSECTION), різницю множин (SETDIFFERENCE), симетричну різницю множин (SYMDIFFERENCE), перевірку, чи є одна множина підмножиною іншої (ISSUBSET).
4. Провести експериментальне визначення часу роботи операцій в залежності від розміру множини.

Якщо ви використовуєте мову програмування, яка не підтримує ООП, для виконання завдання розробіть систему структур даних та відповідних функцій роботи із ними. Ви можете використовувати готові реалізації базових структур даних (масиви, списки, дерева, але не `BitSet` та аналогічні шайтан-структури).

Варіанти завдань наведено нижче.

Варіант 1. Реалізація множин за допомогою списків. Елементи множин можуть бути довільними та різнорідними (необмежений універсум).

Варіант 2. Реалізація множин за допомогою впорядкованих списків (або дерев). Універсумом у даному варіанті виступає (природно впорядкована) множина усіх можливих чисел із плаваючою точкою, яку підтримує ваша мова програмування.

Варіант 3. Реалізація множин за допомогою двійкових векторів. Рекомендується використовувати для представлення масиви 64-бітових регістрів (зазвичай тип `Integer`) розміру не менш ніж $t > 1000$; конкретний розмір може бути параметром ініціалізації. Відповідно, універсумом в даному варіанті буде виступати множина цілих чисел від 0 до $2^{64t} - 1$.

Варіант 4. Реалізація множин за допомогою типу `SPARSESET`. Універсумом виступає множина цілих невід'ємних чисел, максимальне значення елементу `maxVal` та максимальна потужність множини `capacity` повинні бути параметрами ініціалізації. Ваша реалізація повинна підтримувати значення `maxVal` та `capacity` не менші за 65536.

Друга частина завдання повинна оцінити швидкість роботи двох операцій в залежності від потужності множини:

- перевірку наявності елементу в множині (SEARCH);
- однієї з операцій UNION, INTERSECTION, SETDIFFERENCE, SYMDIFFERENCE на ваш розсуд.

Зауважимо, що для коректної оцінки часу виконання необхідно орієнтуватись на такі вимоги:

1) Замір часу необхідно виконувати на випадкових даних, і для кожного експерименту та фіксованих значеннях параметрів необхідно провести щонайменше 1000 експериментів; ітогове значення часу є усередненням від усіх замірів.

2) Дані для експериментів (тобто, випадкові множини заданого розміру) необхідно згенерувати заздалегіть, щоб час генерування не включався у замір.

3) Для аналізу операції SEARCH доцільно розглядати окремо випадки пошуку елементу, який є у множині, та елементу, якого у множині нема.

4) Одержані результати зручно подавати у вигляді графіка «Розмір множини — Час виконання операції».

Після виконання завдання напишіть **коротенький** звіт (1-2 сторінки), в якому опишіть особливості реалізації вашого типу даних та аргументацію вибору вами того чи іншого проєктного рішення, а також наведіть результати ваших обчислювальних експериментів, їх аналіз та інтерпретацію. Код вашого практикуму повинен бути завантажений на платформу GitHub у *відкритий* репозиторій, посилання на яке повинно бути у вашому звіті.